

ID2221 Data-Intensive Computing

Lab assignment Week 2

Benchmark Report

Alexía Margrét Axelsdóttir
Gudný Eva Eiríksdóttir
Kinga Koltai

September 20, 2026

Introduction

In this report, we evaluate the performance improvement associated with four distinct optimization strategies, namely caching, partition pruning, broadcast joins and Adaptive Query Execution on the integrated taxi trips delta table.

Testing Outline

For each experiment, the optimized execution was compared with a corresponding baseline execution. The returned results were also compared to ensure that the optimization did not change the query output. Execution times were measured using Python’s `time.perf_counter()`, with the measurement covering the execution of the Spark query and collection of the results. Where appropriate, a warmup execution was performed before measuring the query execution time to reduce the influence of initial Spark startup and execution overhead.

Instead of applying every optimization to every query, each of the four optimization strategies were mapped to one or more queries where the strategy could have a meaningful effect. Table 1 shows this mapping, while the motivation is presented in section Task 3 of the Week 2 Lab report.

Caching	Query 1-6
Partition pruning	Query 6
Broadcast joins	Query 2
AQE	Query 1

Table 1: Mapping of optimization strategies to queries

Benchmark Results & Performance Discussion

Caching

First, the six queries were executed sequentially without caching, and their total execution time was recorded. The integrated dataset was then cached in Spark memory, after which the same six queries were executed again. This allows the second set of queries to reuse the cached representation instead of repeatedly reading the underlying Delta data.

The results in Output 1 show a significant improvement of execution time with caching for each query individually, as well as the total execution time.

```
===STRATEGY 1: CACHING===
```

```
Query 1: 8.66 seconds
```

```
Query 2: 5.99 seconds
```

```
Query 3: 7.64 seconds
```

```
Query 4: 8.84 seconds
```

```
Query 5: 5.52 seconds
```

```
Query 6: 7.38 seconds
```

```
Query 1: 6.32 seconds
```

```
Query 2: 1.96 seconds
```

```
Query 3: 4.66 seconds
```

```
Query 4: 5.74 seconds
```

```
Query 5: 3.58 seconds
```

```
Query 6: 3.45 seconds
```

```
Results are identical.
```

```
Total execution time without caching: 44.02 seconds
```

```
Total execution time with caching: 25.70 seconds
```

Output 1: Execution times with and without caching

Partition pruning

The integrated taxi dataset is partitioned by day using the `pickup_date` column, so partition pruning is particularly relevant for queries that filter on this column. Although query 6 does filter on the date, the integrated taxi trips table contains few rows outside the range 2024-01-01 to 2024-03-31. This explains why the results in Output 2 do not show a significant difference of execution time with or without partition pruning.

```
===STRATEGY 2: PARTITION PRUNING===  
WITHOUT partition pruning: 4.27 seconds  
WITH partition pruning: 4.23 seconds
```

Results are identical.

Output 2: Execution times with and without partition pruning

Using EXPLAIN FORMATTED (as seen in Output 3 we can see that the version with partition pruning has PartitionFilters set as `[isnotnull(pickup_date#43), (pickup_date#43 >= 2024-01-01), (pickup_date#43 < 2024-04-01)]` whereas the non-pruned version does not. This guarantees that partition pruning is happening, despite the insignificant difference in execution times.

```
== Physical Plan ==  
AdaptiveSparkPlan (8)  
+- Sort (7)  
  +- Exchange (6)  
    +- HashAggregate (5)  
      +- Exchange (4)  
        +- HashAggregate (3)  
          +- Project (2)  
            +- Scan parquet (1)
```

(1) Scan parquet

Output [7]: [pickup_time_local#4, dropoff_time_local#5, passenger_count#6L, trip_distance#7, total_amount#19, airport_fee#21, pickup_date#43]

Batched: true

Location: PreparedDeltaFileIndex [file:/C:/.../labs/Week1/delta/integrated_taxi_trips]

PartitionFilters: [isnotnull(pickup_date#43), (pickup_date#43 >= 2024-01-01), (pickup_date#43 < 2024-04-01)]

ReadSchema: struct<pickup_time_local:timestamp_ntz,dropoff_time_local:timestamp_ntz, passenger_count:bigint,trip_distance:double,total_amount:double,airport_fee:double>

Output 3: EXPLAIN FORMATTED output with partition pruning (excerpt)

Broadcast joins

Query 2 was used to evaluate broadcast joins because it joins the integrated taxi dataset with the relatively small weather condition code lookup table. The baseline query performs the join with `spark.sql.autoBroadcastJoinThreshold` set to -1 to disable automatic broadcasting, while the optimized version uses a broadcast join hint for the smaller lookup table. The results presented in Output 4 show a substantial reduction in execution time. This improvement indicates that avoiding the redistribution of the larger dataset during the join was beneficial for this workload.

===OPTIMIZATION STRATEGY 3: BROADCAST JOINS===

WITHOUT broadcast: 6.73 seconds

WITH broadcast: 1.68 seconds

Results are identical.

Output 4: Execution times with and without broadcast joins

The EXPLAIN FORMATTED outputs (Output 5) shows the difference in execution plans. Without the broadcast hint, Spark uses two **SortMergeJoin** operators. This requires the data on both sides of each join to be repartitioned using **Exchange** operators and sorted before the join can be performed. In contrast, the optimized plan uses two **BroadcastHashJoin** operators. The second version using broadcast joins allows the joins to be performed without repartitioning and sorting the large integrated taxi dataset. The optimized plan therefore substantially reduces the amount of data that needs to be shuffled during the joins, contributing to the speedup we observed.

```

== Physical Plan ==
AdaptiveSparkPlan (26)
+- Sort (25)
  +- Exchange (24)
    +- HashAggregate (23)
      +- Exchange (22)
        +- HashAggregate (21)
          +- Project (20)
            +- SortMergeJoin LeftOuter (19)
              :- Sort (14)
                : +- Exchange (13)
                :   +- Project (12)
                :     +- SortMergeJoin LeftOuter (11)
                :       :- Sort (6)
                :         : +- Exchange (5)
                :         :   +- InMemoryTableScan (1)
                :         :     +- InMemoryRelation (2)
                :         :       +- * ColumnarToRow (4)
                :         :         +- Scan parquet (3)
                :       +- Sort (10)
                :         +- Exchange (9)
                :           +- Filter (8)
                :             +- Scan csv (7)
              +- Sort (18)
                +- Exchange (17)
                  +- Filter (16)
                    +- Scan csv (15)

== Physical Plan ==
AdaptiveSparkPlan (20)
+- Sort (19)
  +- Exchange (18)
    +- HashAggregate (17)
      +- Exchange (16)
        +- HashAggregate (15)
          +- Project (14)
            +- BroadcastHashJoin LeftOuter BuildRight (13)
              :- Project (9)
                : +- BroadcastHashJoin LeftOuter BuildRight (8)
                :   :- InMemoryTableScan (1)
                :     : +- InMemoryRelation (2)
                :     :   +- * ColumnarToRow (4)
                :     :     +- Scan parquet (3)
                :   +- BroadcastExchange (7)
                :     +- Filter (6)
                :       +- Scan csv (5)
            +- BroadcastExchange (12)
              +- Filter (11)
                +- Scan csv (10)

```

Output 5: EXPLAIN FORMATTED output without broadcast joins and with broadcast joins, respectively (excerpt) ⁵

Adaptive Query Execution (AQE)

The query was first executed with AQE disabled and then with AQE enabled. The execution plans and execution times were recorded for both configurations. The results seen in Output 6 show a reduction in execution times when AQE is enabled.

```
=== OPTIMIZATION STRATEGY 4: ADAPTIVE QUERY EXECUTION ===
AQE OFF: 8.22 seconds
AQE ON:  5.53 seconds
```

Results are identical.

Output 6: Execution times with and without AQE

Additionally, the EXPLAIN FORMATTED output (Output 7) shows `AdaptiveSparkPlan` enabled and `isFinalPlan=false`, indicating that AQE is enabled and can modify the physical plan at runtime. However, EXPLAIN FORMATTED shows the preliminary physical plan rather than the final adaptive plan after execution. To see which (if any) optimizations Spark took at runtime, we must look at the physical plan used during execution. The output of `executedPlan()` is presented in Output 8 and shows how the final plan deviated from the initial plan.

```
== Physical Plan ==
AdaptiveSparkPlan (11)
+- Sort (10)
   +- Exchange (9)
      +- HashAggregate (8)
         +- Exchange (7)
            +- HashAggregate (6)
               +- Project (5)
                  +- InMemoryTableScan (1)
                     +- InMemoryRelation (2)
                        +- * ColumnarToRow (4)
                           +- Scan parquet (3)

(...)
(11) AdaptiveSparkPlan
Output [3]: [pickup_month#23423, pickup_zone#15832, num_trips#23424L]
Arguments: isFinalPlan=false
```

Output 7: EXPLAIN FORMATTED output with AQE (excerpt)

```

AdaptiveSparkPlan isFinalPlan=true
+- == Final Plan ==
  ResultQueryStage 3
    +- *(3) Sort [num_trips#20766L DESC NULLS LAST, pickup_month#20765 ASC NULLS FIRST,
      pickup_zone#15832 ASC NULLS FIRST], true, 0
      +- AQEShuffleRead coalesced
        +- ShuffleQueryStage 2
          +- Exchange rangepartitioning(num_trips#20766L DESC NULLS LAST,
            pickup_month#20765 ASC NULLS FIRST, pickup_zone#15832 ASC NULLS FIRST, 200),
            ENSURE_REQUIREMENTS, [plan_id=5861]
            +- *(2) HashAggregate(keys=[_groupingexpression#21208, pickup_zone#15832],
              functions=[count(1)], output=[pickup_month#20765, pickup_zone#15832,
              num_trips#20766L])
              +- AQEShuffleRead coalesced
                +- ShuffleQueryStage 1
                  +- Exchange hashpartitioning(_groupingexpression#21208,
                    pickup_zone#15832, 200), ENSURE_REQUIREMENTS, [plan_id=5833]
                    (...)
        +- == Initial Plan ==
          Sort [num_trips#20766L DESC NULLS LAST, pickup_month#20765 ASC NULLS FIRST,
            pickup_zone#15832 ASC NULLS FIRST], true, 0
          +- Exchange rangepartitioning(num_trips#20766L DESC NULLS LAST, pickup_month#20765
            ASC NULLS FIRST, pickup_zone#15832 ASC NULLS FIRST, 200), ENSURE_REQUIREMENTS,
            [plan_id=5785]
            +- HashAggregate(keys=[_groupingexpression#21208, pickup_zone#15832],
              functions=[count(1)], output=[pickup_month#20765, pickup_zone#15832,
              num_trips#20766L])
              +- Exchange hashpartitioning(_groupingexpression#21208, pickup_zone#15832, 200),
                ENSURE_REQUIREMENTS, [plan_id=5782]
                (...)

```

Output 8: The executed plan with AQE (excerpt)